

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería



Frameworks

Link del documento: [Frameworks](#)

Genser Catalán 23401, Fernando Ruíz 23065, Hugo Barillas, Jose López 23773

Link repositorio GITHUB: <https://github.com/JosFer720/GS-Stock-y-Facturacion>

Link

CANVA:https://www.canva.com/design/DAGhjecz1vc/FyDT369EJfX2poT_o1GRgw/edit?utm_content=DAGhjecz1vc&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton

Erick Marroquín

Ingeniería en Software 1 -- CC3090

Guatemala, 2025

ExpresJS

1. Nombre y descripción general incluyendo propósito y qué área del desarrollo de aplicaciones resuelve.

Express.js es un framework web de código abierto para Node.js que se caracteriza por su enfoque minimalista y flexible. Su principal propósito es facilitar el desarrollo del lado del servidor, permitiendo a los desarrolladores construir aplicaciones web y APIs robustas y escalables. Gracias a su sencillez y eficiencia, Express.js se utiliza ampliamente en la creación de servicios RESTful, aplicaciones en tiempo real y sistemas empresariales, simplificando la gestión de rutas y peticiones HTTP (MDN Web Docs, n.d.).

2. Principios de funcionamiento y componentes.

El funcionamiento de Express.js se basa en un diseño modular centrado en el uso de middleware y rutas. Las rutas determinan cómo responde la aplicación a solicitudes HTTP específicas, definiendo los endpoints y los métodos (GET, POST, PUT, DELETE, etc.). Por otro lado, los middleware son funciones que se ejecutan en el transcurso del ciclo de vida de una solicitud, permitiendo realizar tareas como el procesamiento de datos, la autenticación, el manejo de errores y la integración de funcionalidades adicionales. Esta arquitectura modular no solo facilita el desarrollo y mantenimiento de la aplicación, sino que también mejora su escalabilidad y adaptabilidad (MDN Web Docs, n.d.).

3. Qué tipo de framework es y cuánto del proceso de desarrollo cubre, si es necesario combinarlo con otras tecnologías.

Express.js es un framework de backend que se centra en la lógica del servidor y el procesamiento de solicitudes HTTP. Debido a su naturaleza minimalista, ofrece únicamente las herramientas esenciales para la configuración y gestión de un servidor web, lo que lo convierte en una base sólida pero que a menudo requiere complementarse con otras tecnologías. En un entorno de desarrollo completo, es común combinar Express.js con frameworks de frontend como Angular, React o Vue para la interfaz de usuario, así como con sistemas de gestión de bases de datos como MongoDB, MySQL o PostgreSQL para el almacenamiento de información. Esta integración permite abordar de manera integral tanto la lógica del servidor como la experiencia del usuario en la aplicación (MDN Web Docs, n.d.).

4. Qué patrones de diseño y arquitectónicos implementa el framework seleccionado y en qué parte de este son implementados.

a) Patrón Middleware

Implementado en su sistema de enrutamiento y procesamiento de solicitudes. Permite agregar funcionalidades como autenticación, manejo de errores y análisis de datos en capas intermedias antes de responder al cliente. Se usa con `app.use()`, `app.get()`, `app.post()`, etc. (Harsh, K, 2023)

b) Arquitectura Basada en MVC (Modelo-Vista-Controlador)

No lo impone, pero facilita su implementación. Separa la lógica de negocio (Modelo), la gestión de solicitudes (Controlador) y la representación de datos (Vista). Útil para proyectos grandes.

c) Patrón Proxy Inverso

Express puede actuar como un proxy para manejar solicitudes y redirigirlas a otros servidores. Se usa en aplicaciones que necesitan balanceo de carga o redirección de tráfico. (Harsh, K,2023)

5. En qué situaciones recomienda su uso.

Desarrollo de APIs RESTful

Ideal para crear servicios backend que proporcionen datos a aplicaciones frontend o móviles.

Aplicaciones web ligeras

Para proyectos donde no se necesita un framework monolítico como Laravel o Django.

Microservicios

Su flexibilidad y compatibilidad con Node.js lo hacen excelente para arquitecturas de microservicios

6. Semejanzas y diferencias de los frameworks seleccionados.

Similitudes:

- Lenguaje: Ambos utilizan JavaScript como lenguaje principal.
- Ecosistema: Tanto Express.js como React forman parte del ecosistema de Node.js

Diferencias:

- Renderizado: Express.js genera contenido HTML en el servidor y lo envía al cliente, mientras que React renderiza componentes en el cliente utilizando el Virtual DOM para actualizaciones eficientes
-

React Native

Nombre y Descripción General

React Native es un framework de desarrollo de aplicaciones móviles creado por Meta. Su principal propósito es permitir el desarrollo de aplicaciones nativas para iOS y Android utilizando JavaScript y React. React Native facilita la creación de interfaces de usuario con

una experiencia de usuario fluida y nativa, sin necesidad de desarrollar dos aplicaciones separadas para cada plataforma.

Principios de Funcionamiento y Componentes

- **Reutilización de Código:** Permite escribir un solo código en JavaScript y compartir la mayor parte entre Android e iOS.
- **Componentes Nativos:** Utiliza componentes nativos para renderizar la interfaz de usuario, logrando un rendimiento cercano al nativo.
- **Bridge de JavaScript:** Permite la comunicación entre el código JavaScript y los componentes nativos de la plataforma.
- **Hot Reloading:** Posibilita ver los cambios en tiempo real sin necesidad de recompilar la aplicación.
- **Ecosistema y Librerías:** Compatible con numerosas librerías de terceros que expanden sus funcionalidades.

Tipo de Framework y Cobertura en el Proceso de Desarrollo

React Native es un framework de frontend para el desarrollo de aplicaciones móviles. Cubre gran parte del proceso de desarrollo, pero en muchos casos es necesario complementarlo con librerías adicionales para la gestión del estado (Redux, MobX), la navegación (React Navigation), y la interacción con APIs nativas (Native Modules).

Patrones de Diseño y Arquitectónicos Implementados

- **MVVM (Model-View-ViewModel):** Ayuda a separar la lógica de la interfaz.
- **Component-Based Architecture:** Facilita la reutilización y mantenimiento del código.
- **Bridge Pattern:** Permite la comunicación entre JavaScript y los componentes nativos de cada plataforma.
- **Flux/Redux:** Mecanismo para la gestión del estado de la aplicación.

Situaciones en las que se Recomienda su Uso

- **Desarrollo de aplicaciones móviles multiplataforma sin perder rendimiento nativo.**

- Empresas que desean reducir costos al mantener un solo equipo de desarrollo.
- Aplicaciones con interfaces dinámicas que requieren actualizaciones frecuentes sin pasar por las tiendas de aplicaciones (Code Push).
- Proyectos que ya usan React en web y desean reutilizar conocimiento y código.

Semejanzas y Diferencias con Otros Frameworks

Característica	React Native	Flutter	Ionic
Lenguaje	Javascript	Dart	HTML, CSS, Javascript
Rendimiento	Nativo	Nativo	Webview
UI	Componentes nativos	Widgets propios	Basado en HTML/CSS

Conclusión

React Native es una solución poderosa para el desarrollo de aplicaciones móviles multiplataforma, ofreciendo un equilibrio entre rendimiento y facilidad de desarrollo. Su ecosistema en constante crecimiento y su integración con React lo hacen una opción atractiva para equipos que buscan desarrollar aplicaciones escalables y eficientes.

Referencias

MDN Web Docs. (n.d.). *Introducción a Express/Node - Aprende desarrollo web*. Recuperado de https://developer.mozilla.org/es/docs/Learn/Server-side/Express_Nodejs/Introduction

Harsh, K. (2023, February 14). *Guía Exhaustiva de Patrones de Diseño de JavaScript*. Kinsta®; Kinsta. <https://kinsta.com/es/blog/patrones-de-diseno-javascript/>

Introduction · React native. (2025, February 19). <https://reactnative.dev/docs/getting-started>

De Imagina, E. (2025, March 14). *Aprende React Native – Tutorial de Primeros Pasos*. Imagina Formación. <https://imaginaformacion.com/tutoriales/aprende-react-native-tutorial-de-primeros-pasos>